

3세션: AI coding agent 시대의 Cloud Native/DevOps 마인드셋 특강

수업 목표

- DevOps/Cloud Engineer가 현업에서 맡는 역할을 코드 작성 너머의 운영 책임으로 설명한다.
- 학습 중 에러와 blocker를 실패가 아니라 증거 수집 기회로 다루는 태도를 적용한다.
- 공식 문서, 실행 증거, 동료 질문, AI coding agent를 함께 사용하는 학습 방식을 이해한다.
- Claude Code, Codex 같은 coding agent가 작은 앱 제작을 쉽게 만들었지만, 비즈니스 서비스화에는 인프라/운영 절차 이해가 필요함을 평가한다.
- AI에게 일을 맡길 때 요구사항, 실행 조건, 보안, 비용, 배포, 관찰, 장애 대응 기준을 사람이 제시해야 함을 설명한다.

시간

15:00~16:00

오늘의 초점

- AI coding agent가 만들어 주는 작은 앱과 실제 비즈니스 서비스의 차이를 구분한다.
- agent output을 검토하기 위한 인프라/운영 기준을 학습한다.
- 실행 조건, 보안, 비용, 관찰, 장애 대응을 사람의 책임으로 정리한다.

50~60분 학습 흐름

시간	활동	내가 확인할 것
15:00~15:06	로드맵 연결: 도구보다 운영 문제	이전 세션의 spine을 AI 시대 질문으로 연결한다.
15:06~15:16	DevOps/Cloud Engineer 역할 설명	현업 역할을 coding, deployment, security, observability, cost로 넓힌다.
15:16~15:28	AI coding agent 변화 설명	작은 앱 제작 속도와 운영 책임 사이의 간극을 보여준다.
15:28~15:40	비즈니스 서비스화 checklist 강의	agent 산출물을 검토할 기준을 구체화한다.
15:40~15:48	blocker rewrite 활동	실패 경험을 증거 기반 질문으로 바꾸는 훈련을 한다.
15:48~15:50	공유와 피드백	좋은 증거와 약한 증거를 구분한다.
15:50~16:00	휴식 및 다음 세션 전환	50분 수업 후 협업/자기소개 세션으로 전환한다.

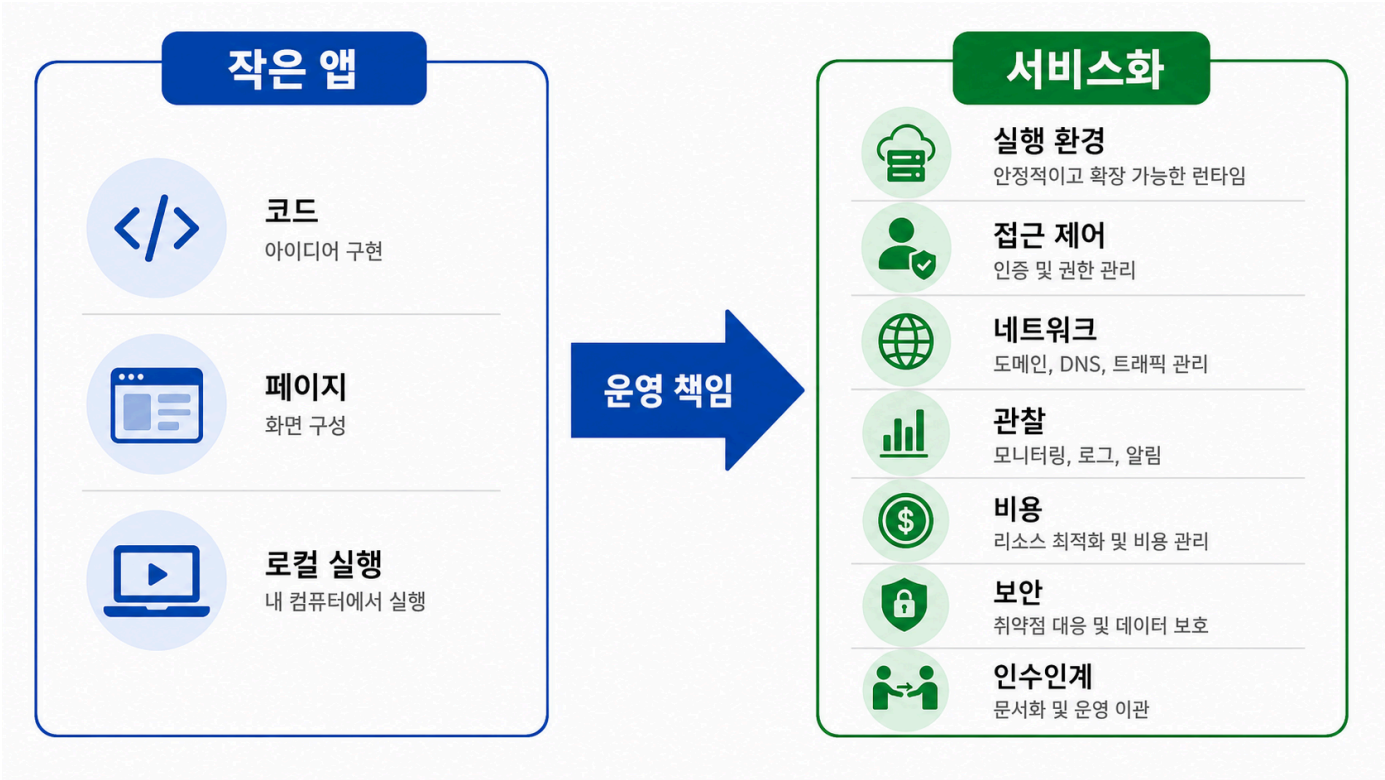
15:00~15:06 로드맵 연결: 도구보다 운영 문제

- 진행: 로드맵 연결: 도구보다 운영 문제
- 초점: 이전 세션의 spine을 AI 시대 질문으로 연결한다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

핵심 설명

- "AI agent가 작은 앱을 만들어 주는 장면은 이미 현실입니다. 하지만 현업의 질문은 '앱이 보이나요?'에서 끝나지 않습니다. 누가 접근할 수 있는지, 어디서 실행되는지, 비용이 얼마인지, 장애가 나면 무엇을 볼지, 되돌릴 수 있는지가 서비스화의 질문입니다."
- "DevOps는 개발자가 운영 일을 떠안는다는 뜻이 아닙니다. 개발, 운영, 보안, 품질, 비즈니스가 같은 증거를 보고 더 빠르고 안전하게 변경하기 위한 방식입니다."
- "AI에게 좋은 답을 받는 사람은 프롬프트를 예쁘게 쓰는 사람이 아니라, 실행 조건과 검증 기준을 정확히 아는 사람입니다. 이 과정에서 여러분은 agent에게 일을 시키는 기준과 agent 결과를 의심하는 기준을 배웁니다."

Visual 1: 작은 앱과 서비스화의 차이



왼쪽은 코드와 화면이 만들어진 상태이고, 오른쪽은 비즈니스에서 운영 가능한 서비스가 되기 위해 필요한 책임이다. 수업에서는 오른쪽 항목을 하나씩 Week 1 spine과 연결한다.

15:06~15:16 DevOps/Cloud Engineer 역할 설명

- 진행: DevOps/Cloud Engineer 역할 설명
- 초점: 현업 역할을 coding, deployment, security, observability, cost로 넓힌다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

Insight: 작은 앱 제작과 비즈니스 서비스화는 다르다

Claude Code와 Codex 같은 coding agent 덕분에 작은 앱을 만드는 일은 훨씬 쉬워졌다. 프론트엔드 화면, 간단한 API, 테스트 초안, README 초안은 초보자도 빠르게 얻을 수 있다. 그러나 "작은 앱이 보인다"와 "비즈니스에서 사용 가능한 서비스다"는 다른 문제다.

단계	사람이 알아야 하는 질문	AI에게 맡길 때 필요한 기준
요구사항	누구의 어떤 문제를 해결하는가?	기능 목록보다 사용자 흐름과 제약을 먼저 쓴다.
실행 조건	어떤 runtime, command, port, data가 필요한가?	start/check/stop evidence를 요구한다.
보안	secret, 권한, 인증, 개인정보는 어떻게 보호되는가?	민감정보 비노출과 권한 범위를 검증한다.
인프라	compute, storage, network, identity는 어디에 있는가?	Docker/AWS/Kubernetes 선택 이유를 설명한다.
배포	변경을 어떻게 build, release, deploy, rollback하는가?	배포 절차와 실패 시 되돌림 기준을 요구한다.
관찰	정상/비정상상을 무엇으로 판단하는가?	log, status, metric, alert 기준을 요구한다.
비용	어떤 리소스가 비용을 발생시키는가?	예상 비용, free-tier 범위, cleanup을 요구한다.
장애 대응	실패하면 누가 무엇을 보고 조치하는가?	RCA, runbook, known issue를 요구한다.

성공 사례로 보는 변화

사례	보여주는 점	수업에서 가져올 교훈
OpenAI Codex cloud/software engineering agent	repository 기반 작업, bug fix, feature 작성, PR 제안처럼 실제 개발 업무를 병렬로 수행하는 방향으로 발전했다.	AI는 코딩 속도를 높이지만, 사람이 task boundary와 review 기준을 줘야 한다.
OpenAI Codex app	여러 agent, worktree, diff review, sandbox permission 같은 엔지니어링 통제를 강조한다.	agent를 쓰는 사람은 변경 검토, 권한, 격리의 의미를 알아야 한다.
Codex harness와 SRE/code reviewer 활용	coding assistant뿐 아니라 code reviewer, SRE agent로 활용할 수 있는 harness 관점도 제시된다.	DevOps 지식이 있으면 AI를 단순 코드 생성기가 아니라 운영 보조 agent로 설계할 수 있다.
Claude Code workflows	코드베이스 이해, 오류 수정, plan mode, pipeline/스크립트 활용을 workflow로 설명한다.	좋은 질문은 reproduce command, stack trace, plan, test, review 기준을 포함한다.
Claude Code subagents	전문 작업을 분리하고 별도 context와 tool 권한으로 처리한다.	보안 리뷰, 테스트, 배포, 문서화 agent를 목적별로 나누는 사고가 필요하다.

핵심 태도

태도	의미	수업 적용
증거 기반	command, log, status, screenshot filename, README로 말한다.	질문할 때 증상과 시도를 함께 쓴다.
작은 단위	큰 시스템을 process, file, port, config로 쪼갬다.	에러를 "전체가 안 됨"이 아니라 실패 지점으로 좁힌다.
공개 가능한 기록	민감정보를 제외하고 동료가 도울 수 있는 맥락을 남긴다.	token, MFA, 결제 정보는 공유하지 않는다.
공식 문서 우선	AI 답변과 블로그는 공식 문서와 실행으로 검증한다.	vendor 문서와 실행 결과를 함께 본다.
blocker 공유	막힌 지점을 숨기지 않고 증상과 시도한 일을 기록한다.	막힌 시간보다 기록 품질을 본다.
절차 이해	AI에게 시키기 전에 필요한 단계와 검증 기준을 말한다.	agent prompt에 완료 조건을 포함한다.
운영 관점	코드 너머 인프라, 배포, 보안, 비용, 관찰 가능성을 함께 본다.	작은 앱을 서비스 checklist로 평가한다.

Visual 2: AI output 검증 경계



읽는 순서: agent가 만든 결과는 바로 완료가 아니다. 조건, 안전, 실행, 인수인계의 네 단계를 지나야 서비스 후보가 된다.

15:16~15:28 AI coding agent 변화 설명

- 진행: AI coding agent 변화 설명
- 초점: 작은 앱 제작 속도와 운영 책임 사이의 간극을 보여준다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

Visual 3: 서비스화 체크리스트

관점	확인할 증거	AI에게 요구할 것
실행 환경	runtime, command, port	start/check/stop 절차
보안	secret 비노출, 권한 범위	민감정보 제외와 권한 설명
비용	리소스 종류, cleanup	예상 비용과 정리 절차
관찰	log, status, metric	정상/비정상 판단 기준
인수인계	README, runbook, known issue	다음 사람이 재현할 문서

15:28~15:40 비즈니스 서비스화 checklist 강의

- 진행: 비즈니스 서비스화 checklist 강의
- 초점: agent 산출물을 검토할 기준을 구체화한다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

활동: blocker rewrite와 service checklist

1. 최근 기술 학습에서 막혔던 경험 하나를 고른다.
2. 증상을 있는 그대로 쓴다. 예: "설치가 안 됐다", "서버가 안 켜졌다".
3. 아래 형식으로 다시 쓴다.

항목	기록
증상	사용자가 본 현상 또는 에러 요약
내가 시도한 것	실행한 절차, 참고한 문서, 바꾼 설정
부족했던 증거	명령어, 경로, 버전, 로그, screenshot 중 빠진 것
다음에는 남길 증거	동료가 재현할 수 있게 남길 정보

4. AI에게 "작은 앱을 만들어줘"라고 요청했을 때 빠질 수 있는 비즈니스 서비스화 항목을 5개 이상 적는다.
5. 짝과 바꿔 보고, "이 기록만 보고 내가 도울 수 있는가?"를 기준으로 피드백한다.

15:40~15:48 blocker rewrite 활동

- 진행: blocker rewrite 활동
- 초점: 실패 경험을 증거 기반 질문으로 바꾸는 훈련을 한다.
- 완료 조건: 이 시간 블록의 결과를 evidence note에 남긴다.

15:48~15:50 공유와 피드백

- 진행: 공유와 피드백
- 초점: 좋은 증거와 약한 증거를 구분한다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

오해 점검

오해	바로잡기
----	------

AI가 코드를 만들면 DevOps 지식은 덜 중요해진다.	코드 생성 속도가 빨라질수록 검증, 배포, 보안, 비용 판단의 중요성이 커진다.
에러를 많이 만나면 실력이 부족한 것이다.	에러를 증거로 바꾸고 재현 가능하게 설명하는 능력이 현업 실력이다.
공식 문서는 전문가만 읽는다.	초급자는 모든 문서를 읽는 것이 아니라 필요한 정의, 제한, 예제를 찾아 검증한다.
agent가 알려준 명령은 그대로 실행해도 된다.	권한, 삭제, 비용, secret 영향이 있는 명령은 실행 전 의미를 확인해야 한다.

15:50~16:00 휴식 및 다음 세션 전환

- 진행: 휴식 및 다음 세션 전환
- 초점: 50분 수업 후 협업/자기소개 세션으로 전환한다.
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

공식/현업 근거

- AWS: What is DevOps? <https://aws.amazon.com/devops/what-is-devops/>
- Google SRE Book: Postmortem Culture <https://sre.google/sre-book/postmortem-culture/>
- OpenAI: Introducing Codex <https://openai.com/index/introducing-codex/>
- OpenAI: Introducing the Codex app <https://openai.com/index/introducing-the-codex-app/>
- OpenAI: Unlocking the Codex harness <https://openai.com/index/unlocking-the-codex-harness/>
- Anthropic Claude Code overview <https://docs.anthropic.com/ko/docs/claude-code/overview>
- Anthropic Claude Code common workflows <https://docs.anthropic.com/ko/docs/claude-code/common-tasks>
- Anthropic Claude Code subagents <https://docs.anthropic.com/id/docs/claude-code/sub-agents>

안내 프롬프트

- 공개 질문: "AI가 만든 작은 앱이 비즈니스 서비스가 되려면 가장 먼저 보강해야 할 책임은 무엇인가?"
- 개인 작성: 내 blocker 문장 1개를 증상, 시도, 필요한 증거 형식으로 다시 쓴다.

산출물

- mindset rewrite note
- AI-to-business insight note
- 비즈니스 서비스화 checklist 초안
- 좋은 질문으로 바꾼 blocker 예시 1개

학술/현업 근거

- AWS DevOps: 개발과 운영의 결합은 문화, 자동화, 측정, 협업을 포함한다.
- Google SRE postmortem culture: 실패를 개인 탓으로 처리하지 않고 재발 방지 학습으로 전환한다.
- Google Cloud DevOps guidance: 빠른 변경은 실행 증거, 자동화, 복구 가능성과 함께 다룰 때 운영 성과로 이어진다.
- ABET-style communication: 문제, 증거, 제약을 타인이 이해할 수 있게 표현한다.
- CS2023 Knowledge/Skill/Disposition: agent 사용 지식, 검증 기술, 책임 있는 태도를 함께 평가한다.

AI coding agent 시대 인사이트

- agent는 junior의 생산성을 높일 수 있지만, junior가 senior의 검토 기준 없이 결과를 배포하면 위험도 같이 빨라진다.
- DevOps 학습은 agent가 만든 코드를 서비스로 만드는 질문 목록을 제공한다.
- 미래의 강한 엔지니어는 agent를 많이 쓰는 사람이 아니라 agent output을 운영 기준으로 검증하고 개선하는 사람이다.

세션 체크리스트

- ☐ 작은 앱 제작과 비즈니스 서비스화의 차이를 사례로 설명했다.
- ☐ 학생이 blocker를 증거 기반 형식으로 다시 썼다.
- ☐ 학생이 서비스화 체크리스트에서 자신이 약한 항목을 표시했다.

다음 연결

다음 세션은 아이스브레이킹과 자기소개를 통해 협업 규칙을 만든다.

평가 기준

기준	2점 evidence
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~6 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.